

Спец Кит (github/spec-kit) — практическое руководство

Как пользоваться инструментом **Спец Кит** (CLI specify) вместе с Claude Code. Источник: официальный README —

<https://github.com/github/spec-kit>, docs — <https://github.github.io/spec-kit/>

Что это (коротко)

Открытый тулkit от GitHub для **Spec-Driven Development (SDD)** — «разработки от спецификации». Идея: **спецификация становится исполняемой** и напрямую порождает реализацию, а не просто «направляет» её. Цель — уйти от «вайбкодинга каждого куска» к предсказуемому процессу: сначала договариваемся *что* и *почему*, потом *как*, потом задачи, потом код.

Принцип работы

Разработка идёт **多**-шагово, не «одним промптом»: 1. **Конституция** — принципы проекта (качество, тесты, UX, перформанс), которым следуют все дальнейшие шаги. 2. **Спека (specify)** — *что* строим (требования, пользовательские истории). Без техстека. 3. **Уточнение (clarify)** — закрыть недосказанное до плана. 4. **План (plan)** — техстек и архитектура. 5. **Задачи (tasks)** — разбить план на исполнимые задачи (с зависимостями и метками параллельности). 6. **Проверка (analyze/checklist)** — сверка артефактов на полноту/согласованность. 7. **Реализация (implement)** — агент выполняет задачи по порядку.

Как это устроено технически

- CLI **specify** бутстрапит проект: кладёт шаблоны и slash-команды в твоего агента (Claude Code).
- Появляется папка **.specify/**: memory/constitution.md (принципы), templates/ (шаблоны spec/plan/tasks), scripts/ (вспомогательные скрипты).
- Каждая фича живёт в **specs/<NNN-feature>/**: spec.md, plan.md, tasks.md, research.md, data-model.md, contracts/, quickstart.md (создаётся отдельная git-ветка, напр. 001-...).

Требования

- ОС: Linux / macOS / Windows.
- **uv** (менеджер пакетов) или **pipx**, **Python 3.11+**, **Git**.
- Поддерживаемый AI-агент — **Claude Code поддерживается** (плюс 30+ других).

Установка и обновление

установка (взять последний тег vX.Y.Z со страницы *Releases*)

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git@vX.Y.Z
```

```
specify self check      # есть ли новая версия (только чтение)
specify self upgrade    # обновить до последней
specify self upgrade --dry-run
```

Инициализация проекта (под Claude Code)

```
specify init my-project --integration claude
cd my-project
```

или в текущей папке:

```
specify init . --integration claude    # (или --here)
specify init . --force --integration claude # если папка не пустая
specify integration list                # список поддерживаемых агентов
```

После init в Claude Code появятся slash-команды /speckit.* — это признак, что всё настроено.

Твой рабочий цикл (по шагам)

Запускаешь Claude Code в папке проекта и по очереди:

```
/speckit.constitution  Принципы: качество кода, тесты, единый UX, перформанс,
безопасность...
/speckit.specify       Что строим (требования и истории), без техстека
/speckit.clarify       Ответить на уточняющие вопросы (до плана)
/speckit.plan          Техстек: NestJS + Vue + Supabase + Prisma... (наши
решения)
/speckit.tasks         Разбить на задачи
/speckit.analyze       (опц.) сверка spec/plan/tasks на согласованность
/speckit.implement     Выполнить задачи и собрать фичу
```

Slash-команды (в агенте)

Основные: /speckit.constitution, /speckit.specify, /speckit.plan, /speckit.tasks, /speckit.implement, /speckit.taskstoissues (задачи — GitHub Issues).

Опциональные (качество): /speckit.clarify (уточнение до плана), /speckit.analyze (сверка артефактов после tasks, до implement), /speckit.checklist (чек-листы качества требований — «юнит-тесты для текста ТЗ»).

CLI-команды specify

- specify init <name|.> [--integration claude] [--here] [--force] [--ignore-agent-tools] — создать/инициализировать проект.
- specify self check | upgrade [--dry-run] [--tag vX.Y.Z] — проверка/обновление CLI.
- specify integration list — список интеграций.
- specify extension search | add <name> — расширения (новые команды/возможности).

- specify preset search | add <name> — пресеты (меняют *как* работают шаблоны/команды).

Кастомизация (когда понадобится)

- **Extensions** — добавляют новые команды/возможности (напр. интеграция с Jira, пост-ревью кода).
- **Presets** — меняют формат спек/планов/задач под ваши стандарты (напр. обязательный security-гейт, терминология, язык).
- **Project-local overrides** — .specify/templates/overrides/ для разовых правок под один проект.
- Приоритет разрешения шаблонов: overrides → presets → extensions → core.

Как это ложится на НАШ проект (важно)

- Наш проект **brownfield** (уже есть решения и общий роадмап). Спец Kit удобнее всего применять **по одной новой фиче**: завёл фичу → specify → clarify → plan → tasks → implement.
- **constitution.md** **наполняем из наших инвариантов**: security-инварианты (security/README.md), white-label (0028), перформанс-бюджеты (performance.md), «модульный монолит» (0024), definition of done (claude-code/instructions.md). Тогда все спеки/планы им следуют.
- **Не путать два plan.md**: наш cwork/plan.md — это высокоуровневый роадмап всего продукта; спец-kit создаёт **свой plan.md/tasks.md на каждую фичу** в specs/<feature>/. Наш = «что делаем в целом», спец-kit = «детальная спека конкретной фичи».
- Техстек в /speckit.plan задаём из наших решений (NestJS + Vue 3 + shadcn-vue + self-hosted Supabase + Prisma + REST/SSE/WS + Docker/Caddy).
- Работает поверх **Claude Code** (наш выбор) — совместимо.

Мини-пример (одна фича)

/speckit.specify Инбокс агента: список тикетов с фильтрами, открытие тикета, статусы и приоритеты...

/speckit.clarify

/speckit.plan NestJS API + Prisma (Supabase Postgres), Vue 3 + shadcn-vue, keyset-пагинация, виртуализация списка

/speckit.tasks

/speckit.analyze

/speckit.implement

Ссылки

- Репозиторий: <https://github.com/github/spec-kit>
- Документация: <https://github.github.io/spec-kit/>
- CLI Reference: <https://github.github.io/spec-kit/reference/overview.html>
- Методология SDD: <https://github.com/github/spec-kit/blob/main/spec-driven.md>